Introduction to Message Passing Interface: Graded Exercise 1

Ali Abdollahian Noghabi | F1_MA_HPC_1-24 | University of Luxembourg | 0261874110

1. Introduction

The goal of the exercise is to compute a linear algebra expression involving two matrices and a vector,

$$\mathbf{d} = \mathbf{A} \cdot \mathbf{B} \cdot \mathbf{c}$$

where **A** is 128000 × 500, **B** is 500 × 1000 and **c** has 1000 elements, all double precision with random values in [-1, 1] (seeds 10, 20 and 30). After that the Euclidean norm $\|\mathbf{d}\|_2$ of the result has to be calculated. The code is parallelized with MPI and was run on the AION cluster with 1, 2, 4, …, 128 MPI processes on one full node, and the timings were collected to study the scaling.

The main computational decision was how to parenthesize the operations. With (**A** · **B**) · **c**, the product **A** · **B** produces a giant intermediate matrix of size 128000 × 1000 (about 1 GB) and needs about 64 billion multiply-add operations. The efficient way is **A** · (**B** · **c**): first the intermediate vector **x** = **B** · **c** (M = 500 elements) is computed, which takes only 500 × 1000 = 500000 operations, and then **d** = **A** · **x** (N = 128000 elements), which takes 128000 × 500 = 64 million operations. This is about 1000 times less work, so this order is used in the code.

2. Explanation of the parallelization

Rank 0 initializes **A**, **B** and **c** with the random generator and the sizes are sent to all ranks with `MPI_Bcast`. To distribute the work across P ranks:

1. **Computing x = B · c. B** (500 × 1000) and **c** (1000) are tiny, so there are two ways to do it. I coded it dynamically with a parameter `bc_parallel`, so both can be tested: (0) rank 0 computes **x** alone, or (1) **c** is sent with `MPI_Bcast`, the 500 rows of **B** are split between the ranks with `MPI_Send`/`MPI_Recv` and each rank computes its piece of **x**. In both cases every rank puts its piece in a zero-filled vector of length M and one `MPI_Allreduce` with `MPI_SUM` gives the full **x** on all ranks (in mode 0 the other ranks just add zeros, so the same line works for both).
2. **Distribute A.** The N = 128000 rows of **A** are split across the P ranks, N_{local} = N/P rows each (if N is not divisible by P, the first ranks get one extra row). Rank 0 sends the blocks with `MPI_Send` and the others receive them with `MPI_Recv`, so each rank holds $\mathbf{A}_{local}$ of size N_{local} × M.
3. **Local computation.** Each rank computes its segment $\mathbf{d}_{local} = \mathbf{A}_{local} \cdot \mathbf{x}$ and the sum of the squares of its local elements, $\Sigma (d_{i,local})^2$.
4. **Global reduction.** `MPI_Reduce` with `MPI_SUM` adds all local sums on rank 0, which takes the square root to obtain $\|\mathbf{d}\|_2$.

For the benchmark, three execution intervals are measured with `MPI_Wtime`, and the maximum over the ranks is reported (the slowest rank decides):

- **Initialization time:** allocation, random generation and distribution of **A** (and of **B** and **c** in mode 1).
- **Matrix operations time:** only the local multiplications **B** · **c** and **A** · **x**.
- **Reduce operations time:** the `MPI_Allreduce` of **x** and the `MPI_Reduce` of the norm.

3. Explanation of the scaling results

Table 1 and Fig. 1 show the results for N = 128000 (S = T(1)/T(P) is the speedup and E = S/P the efficiency; "tot" is the total time, "calc" the matrix operations time). The columns S_{tot} to E_{calc} are for mode 1 (**B** · **c** in parallel); the last column is S_{calc} for mode 0 (**B** · **c** on rank 0). The three times are maxima over the ranks taken separately, so they do not add up exactly to the total.

*Table 1: Timings, speedup and efficiency on AION (mode 1, **B** · **c** in parallel).*

P	Init [s]	Calc [s]	Reduce [s]	Total [s]	S_{tot}	S_{calc}	E_{calc}	S_{calc} (rank 0)
1	0.2828	0.0627	0.0001	0.3456	1.00	1.00	1.00	1.00
2	0.3452	0.0309	0.0002	0.3762	0.92	2.03	1.02	1.99
4	0.3761	0.0154	0.0588	0.3916	0.88	4.08	1.02	3.96
8	0.3794	0.0077	0.0773	0.3873	0.89	8.17	1.02	7.72
16	0.3796	0.0041	0.0899	0.3838	0.90	15.39	0.96	14.20
32	0.3986	0.0025	0.1044	0.4014	0.86	25.51	0.80	22.87
64	0.4036	0.0025	0.1183	0.4063	0.85	25.02	0.39	21.80
128	0.4738	0.0015	0.1462	0.4756	0.73	40.98	0.32	34.82

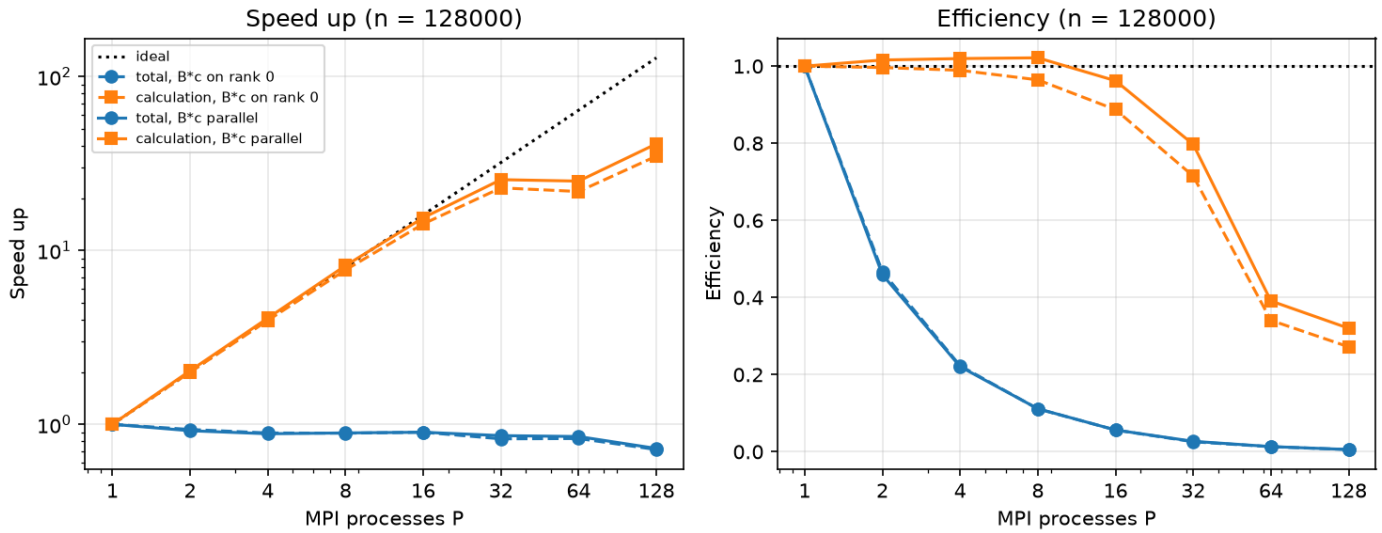


*Figure 1: Speedup and efficiency (solid: **B** · **c** parallel, dashed: **B** · **c** on rank 0).*

Total time. It does not scale: it goes from 0.346 s at P = 1 to 0.476 s at P = 128, so S_{tot} = 0.73 and the efficiency goes to almost zero. Fig. 2 shows why: most of the time is initialization (82% at P = 1). Rank 0 alone generates the 64 million random numbers of **A** and sends the blocks one by one to the other ranks, so this part is serial and even grows with P (0.28 s to 0.47 s). With 82% serial fraction, Amdahl's law says the total speedup could never be more than about 1.2.

Matrix operations. These scale well. The calculation follows the ideal line up to 16 processes (S_{calc} = 15.4 at P = 16; the values slightly above ideal for P = 2 to 8 are probably cache effects and timing noise). After that it drops: E_{calc} = 0.80 at P = 32, 0.39 at P = 64 and 0.32 at P = 128 (S_{calc} = 41). At P = 128 each rank works only about 1.5 ms, and a matrix-vector product reads each element of **A** only once, so it is probably limited by the memory bandwidth of the node, plus the timing noise and the small serial part.

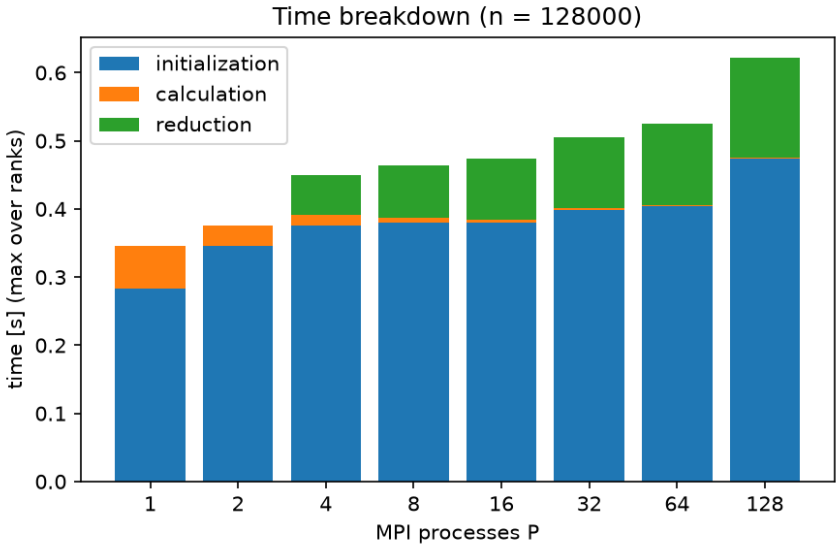


Figure 2: Time breakdown (maximum over ranks, mode 1).

Reduce operations. The time is 0.0002 s at P = 2 but jumps to about 0.06 s at P = 4 and grows to 0.15 s at P = 128. The messages are tiny (500 doubles and one double), so this is not a bandwidth effect. I think it is mostly waiting time: the `MPI_Allreduce` is the first point where the ranks synchronize, and the ranks that got their block of **A** early have to wait for the others.

B · c in parallel or on rank 0. In the plots the two versions are almost the same. The parallel version is better in the calculation time from P = 32 (S_{calc} = 41.0 against 34.8 at P = 128), because computing **x** on one rank leaves a small serial part of a fraction of a millisecond. But for the total time the difference is only a few ms and not systematic (rank 0 is faster for P ≤ 16, parallel for P ≥ 32), so it is within the noise. To really improve the total time the initialization must be parallelized, for example with each rank generating its own rows of **A**, but this changes the random numbers used.